

2.1 OS Objectives and Functions

Definition

Operating system: a program that controls application execution and acts as an interface between applications and computer hardware.

Three Objectives

- Convenience** makes the computer easier to use.
- Efficiency** uses CPU, memory, I/O, files, storage, and network resources efficiently.
- Evolution** permits new functions, tests, fixes, and hardware support without disrupting service.

Two Main Roles

- User/computer interface:** hides hardware details and provides usable services.
- Resource manager:** allocates processor, memory, I/O devices, files, storage, and network resources.

OS Services

- Program development** editors, debuggers, compilers, libraries.
- Program execution** loads code/data, initializes files, I/O, and resources.
- I/O access** uniform device interface, e.g. `read()`, `write()`.
- File access** file organization, permissions, protection.
- System access** login, authorization, resource contention control.
- Error handling** detect hardware/software errors; terminate, retry, or report.
- Accounting** collect usage/performance data; support billing and tuning.

Interfaces: ISA vs ABI vs API

ISA	CPU instruction set	hardware/software boundary; user ISA vs system ISA.
ABI	binary compatibility	system-call interface, registers, calling convention, binary format.
API	source-level calls	library functions used by programmers, e.g. <code>open()</code> , <code>malloc()</code> .

Memory hook: API = coding; ABI = compiled binary execution; ISA = CPU instructions.

Kernel, Control, Evolution

Kernel / Nucleus

Kernel: central OS part, normally resident in main memory, containing the most frequently used OS functions: process, memory, I/O, file, interrupt, and system-call handling.

How the OS Controls the CPU

The OS is also software. It gains control, schedules and allocates resources, gives control to user programs, then regains control through interrupts, system calls, or exceptions.

Why OSs Must Evolve

- Hardware upgrades:** new processors/devices require new support.
- New services:** users and administrators need new tools and features.
- Fixes:** bugs must be repaired without destabilizing old services.

System Calls and Modes

System Calls

System call: controlled entry point for a user program to request OS services such as file I/O, process creation, memory allocation, communication, and protection.

Dual-Mode Operation

- User mode** low privilege; applications cannot directly execute privileged instructions.
- Kernel mode** high privilege; OS kernel performs protected operations.

Flow: a user program requests a system call/trap; the kernel performs the service and returns to user mode.

Exam Questions

Review 2.1

Q: What are three objectives of an OS design?

A: Convenience, efficiency, and ability to evolve.

Review 2.2

Q: What is the kernel of an OS?

A: The core OS part, usually resident in main memory, containing the most frequently used OS functions.

Problem 2.4

Q: What is the purpose of system calls, and how do they relate to dual-mode operation?

A: System calls let user programs request OS services safely. They transfer control from user mode to kernel mode so the OS can perform privileged operations, then return control to the user program.

Problem 2.5

Q: Why might SRM check whether page frames are referenced or changed?

A: To measure real-memory activity. Long-untouched frames are good replacement or reallocation candidates; frequently touched frames indicate memory pressure or possible thrashing, so SRM may reduce multi-programming, swap out address spaces, or adjust memory allocation.

Review 2.6

Q: List five storage-management responsibilities of a typical OS.

A: Process isolation; automatic allocation and management; modular-program support; protection and access control; long-term storage management.

Fast Recall

- OS = application controller + hardware interface + resource manager.
- OS hides hardware details behind services and interfaces.
- Important services: development, execution, I/O, files, access control, errors, accounting.
- Kernel holds common privileged OS functions in memory.
- System calls are the legal bridge from user mode to kernel mode.
- Evolution needs modular design, clear interfaces, and maintainability.